



ARQUITETURA DE SISTEMAS

AULA 1 - INTRODUÇÃO



POR QUE ARQUITETURA DE SOFTWARE?

- ❑ Carreira valorizada: Arquiteto de software aparece no topo das listas de melhores empregos em TI, evidenciando a alta demanda e o prestígio do papel.
- ❑ Ausência de trajetória definida: Ao contrário de profissões tradicionais (médico, engenheiro), não existe um caminho claro para se tornar arquiteto de software, reflexo da falta de consenso sobre seu papel exato.



DESAFIOS PARA DEFINIR ARQUITETURA

- ❑ Falta de definição clara: A indústria não possui uma definição unânime de arquitetura de software. Mesmo especialistas evitam defini-la; Martin Fowler, por exemplo, citou a frase famosa de Ralph Johnson: “Arquitetura é sobre as coisas importantes... seja lá o que for isso”, destacando a subjetividade do termo.
- ❑ Escopo amplo e em expansão: O papel do arquiteto evoluiu além de aspectos puramente técnicos (componentes, padrões). Novos estilos e paradigmas (como microsserviços) ampliaram responsabilidades para incluir preocupações operacionais, de negócio e soft skills.



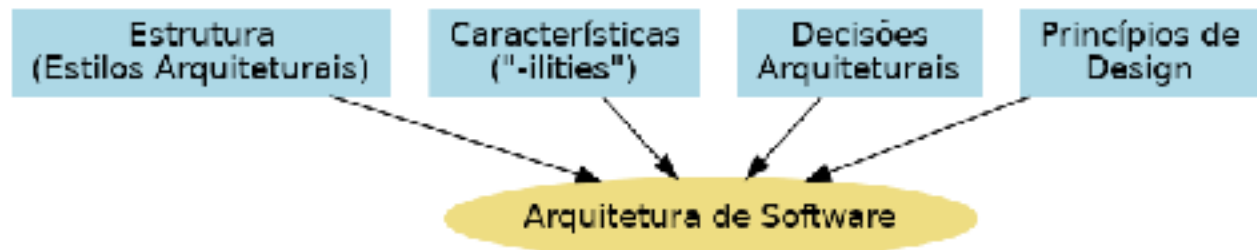
DESAFIOS PARA DEFINIR ARQUITETURA

- ❑ Alvo em movimento: A tecnologia muda rápido; definições antigas ficam obsoletas. Ex.: antes dizia-se que arquitetura envolve escolhas estruturais custosas de mudar, mas com microsserviços tornou-se viável alterar componentes de forma incremental. Arquitetura moderna é dinâmica e evolutiva, não algo estático.
- ❑ Aprendizados do passado: Muito do material clássico de arquitetura reflete soluções da sua época, muitas hoje superadas. Acrônimos e padrões antigos (p.ex. CORBA, CASE, EJB) podem não se aplicar no contexto atual. Entender a história e lições de fracassos passados é importante, mas arquiteturas devem ser adaptadas ao contexto presente.



DEFINIÇÃO DE ARQUITETURA DE SOFTWARE

- ❑ Estrutura do sistema: Organização dos componentes e módulos conforme um ou mais estilos arquiteturais adotados (p.ex., arquitetura em camadas, microsserviços, microkernel, orientada a eventos, etc.).
- ❑ Características arquiteturais: Atributos de qualidade (as “ilities”) que o sistema deve atender. Desempenho (performance), escalabilidade, segurança, disponibilidade, manutenibilidade, entre outros. Geralmente independentes da função do sistema.
- ❑ Decisões arquiteturais: Escolhas e regras de alto nível feitas pelo arquiteto para guiar a construção do sistema. Definem padrões, restrições e diretrizes sobre o que é ou não permitido na solução.
- ❑ Princípios de design: Diretrizes gerais de projeto que orientam as decisões de design de baixo nível. Representam boas práticas recomendadas (não obrigatórias) para manter consistência e qualidade na implementação.



ESTRUTURA (ESTILO ARQUITETURAL)

- ❑ Estilo arquitetural: Padrão macro de organização do sistema. Exemplos: arquitetura em camadas (layered), cliente-servidor, microsserviços, orientada a eventos, microkernel, entre outros. Cada estilo define responsabilidades dos componentes e como eles interagem.
- ❑ Descrição parcial: Indicar somente o estilo usado (“arquitetura em microsserviços”, “sistema em camadas”) não descreve a arquitetura completa. É preciso combinar estrutura + decisões, características e princípios para entender o sistema por inteiro.

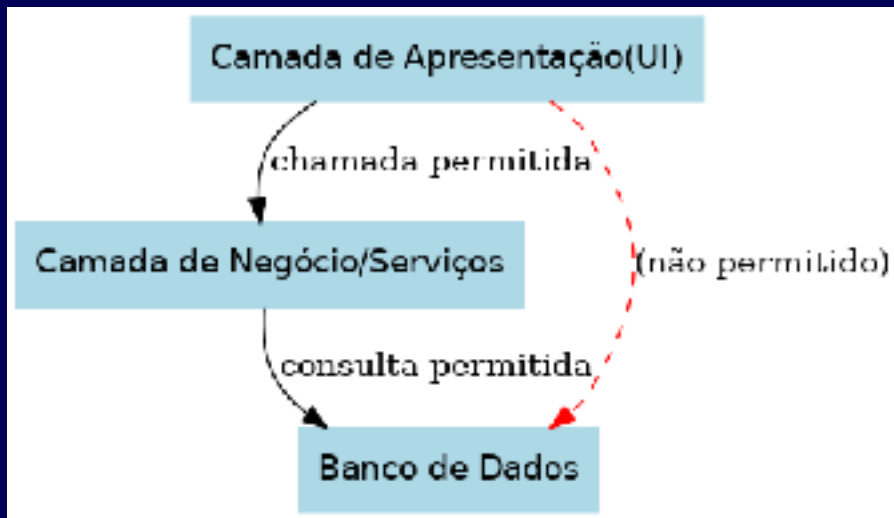


CARACTERÍSTICAS ARQUITETURAIS (“-ILITIES”)

- ❑ Qualidades “-ilities”: Também chamadas de requisitos não-funcionais. Incluem atributos de qualidade como desempenho (performance), escalabilidade, segurança, disponibilidade, confiabilidade, manutenibilidade, usabilidade, entre outros — geralmente terminados em “-idade” (ou “-ility” em inglês).
- ❑ Critérios de sucesso: Essas características definem quão bem o sistema atende às expectativas de qualidade. São ortogonais às funcionalidades (isto é, não dependem do o que o sistema faz, e sim quão bem ele o faz) e muitas vezes determinam a satisfação do usuário e do negócio em produção.



DECISÕES ARQUITETURAIS



- ❑ Regras e restrições: Decisões de arquitetura definem regras que direcionam a implementação. São escolhas de alto nível tomadas para impor padrões e limites — guiam as equipes sobre o que é permitido ou preferido na construção do sistema.
- ❑ Exemplo (camadas): Em uma arquitetura em camadas, pode-se decidir que somente as camadas de negócio/serviços acessam diretamente o banco de dados. A camada de apresentação (UI) fica proibida de acessá-lo, devendo passar pela camada abaixo. Isso garante isolamento entre UI e dados, facilitando manutenções e evitando violações de consistência.
- ❑ Exceções controladas: Decisões arquiteturais devem ser documentadas e comunicadas. Se uma regra precisar ser quebrada em determinado caso, o arquiteto pode conceder uma exceção (uma “variance”), tipicamente através de um processo formal de revisão arquitetural, para avaliar o impacto antes de aprovar a quebra da regra.

PRINCÍPIOS DE DESIGN

- ❑ Boas práticas recomendadas: Princípios de design são orientações gerais para o desenho de componentes e código, em vez de regras rígidas. Exemplos: baixo acoplamento e alta coesão entre classes, preferir composição em vez de herança, programar voltado a interfaces (abstrações) ao invés de implementações concretas, aplicar padrões de projeto (Design Patterns como Factory, Strategy, etc.) onde apropriado.
- ❑ Exemplo (comunicação assíncrona): Em microsserviços, adotar como princípio preferir comunicação assíncrona entre serviços (via mensageria, eventos) para melhorar desempenho e escalabilidade. Contudo, os desenvolvedores têm liberdade para usar chamadas REST/gRPC síncronas se um caso específico exigir. O princípio orienta a escolha preferencial, mas não proíbe alternativas quando justificadas.



O PAPEL DO ARQUITETO DE SOFTWARE

- ❑ Varia conforme o contexto: O cargo “arquiteto de software” não é definido de forma uniforme em todas organizações. Pode ir desde um desenvolvedor líder experiente (focado em decisões técnicas de um projeto) até alguém definindo a estratégia tecnológica de toda a empresa (nível enterprise).
- ❑ Foco em expectativas: Em vez de tentar uma definição rígida do papel, é mais útil pensar nas responsabilidades e expectativas centrais colocadas sobre um arquiteto. Certas competências são esperadas de arquitetos em geral, independentemente do título ou descrição de cargo formal.



EXPECTATIVAS DO ARQUITETO

- ❑ Definir decisões de arquitetura: Estabelecer as decisões arquiteturais e princípios de design que irão guiar as escolhas tecnológicas da equipe, do departamento ou da empresa.
- ❑ Analisar a arquitetura continuamente: Avaliar constantemente a arquitetura vigente e o ambiente tecnológico, recomendando melhorias para evitar degradação estrutural e manter o sistema saudável (vitalidade da arquitetura).
- ❑ Manter-se atualizado: Acompanhar as últimas tendências de tecnologia e do setor, antecipando mudanças. Decisões arquiteturais tendem a durar anos; estar alinhado com o estado da arte ajuda a tomar decisões acertadas e sustentáveis.
- ❑ Garantir conformidade: Assegurar que as equipes de desenvolvimento sigam as decisões arquiteturais e padrões definidos. Monitorar e comunicar desvios, pois violações podem comprometer as características de qualidade esperadas do sistema.



EXPECTATIVAS DO ARQUITETO

- ❑ Experiência diversificada: Ter exposição a múltiplas tecnologias, linguagens, frameworks e plataformas. Um arquiteto eficaz desenvolve conhecimento amplo (generalista) além de sua especialidade, para poder integrar sistemas heterogêneos e avaliar soluções com visão ampla.
- ❑ Conhecimento do domínio de negócio: Entender o setor de atuação (finanças, saúde, varejo, etc.) e o vocabulário do negócio. Isso permite ao arquiteto captar requisitos implícitos, comunicar-se bem com stakeholders não técnicos e propor soluções alinhadas às necessidades reais.
- ❑ Habilidades interpessoais: Demonstrar liderança, comunicação e trabalho em equipe. O arquiteto deve saber liderar tecnicamente os times, mentorando desenvolvedores, mediando decisões e inspirando confiança. Também precisa traduzir ideias complexas em termos claros para diferentes audiências.
- ❑ Navegação política: Lidar com a política organizacional e negociação. Decisões arquiteturais impactam múltiplas equipes e interesses; quase sempre encontram resistência inicial. O arquiteto deve articular os benefícios, negociar compromissos e obter adesão da gerência e times, em vez de impor decisões unilateralmente.



ARQUITETURA E PRÁTICAS DE ENGENHARIA

- ❑ Processo vs. práticas: Historicamente, arquitetura era vista como independente do processo de desenvolvimento (metodologia). No entanto, práticas modernas de engenharia (integração contínua, testes automatizados, entrega contínua) influenciam diretamente a viabilidade de certas arquiteturas. Processos ágeis e iterativos tendem a se alinhar melhor com a natureza evolutiva da arquitetura do que processos rígidos (Waterfall ou cascata).
- ❑ Evolução das práticas: Movimentos como Extreme Programming (XP) focaram em práticas de qualidade (teste em primeiro lugar, refatoração contínua, builds frequentes) independentemente do processo gerencial. Isso evoluiu para Continuous Delivery e posteriormente para a cultura DevOps, que integra desenvolvimento e operações. Arquitetos hoje devem considerar e aproveitar essas práticas de engenharia para viabilizar arquiteturas evolutivas (que conseguem mudar e se adaptar continuamente).



ARQUITETURA E OPERAÇÕES (DEVOPS)

- ❑ Integração com Ops (infraestrutura): No passado, operações (provisão de servidores, deploy, monitoramento) era um setor separado, muitas vezes terceirizado, com pouca influência do desenvolvimento. Arquiteturas eram projetadas de forma defensiva, assumindo restrições da infraestrutura (ex.: incluir lógica complexa de escalonamento dentro da aplicação, pois obter novos servidores era difícil e caro).
- ❑ DevOps e mudança de paradigma: Com o movimento DevOps, desenvolvimento e operações trabalham em conjunto. Arquiteturas modernas como microsserviços transferem muitas preocupações (escalabilidade elástica, tolerância a falhas, monitoramento) para a infraestrutura automatizada (nuvem, orquestração de contêineres). Isso simplifica o design das aplicações, mas exige colaboração: arquitetos entendem das capacidades de Ops/Cloud, e profissionais de Ops usam práticas de software (infraestrutura como código, automação). Juntos alcançam soluções mais flexíveis e escaláveis.



ARQUITETURA E DADOS

- ❑ Decisões de dados: A arquitetura de software tem interseção com a arquitetura de dados. Escolhas como usar banco de dados relacional ou NoSQL, schema centralizado ou distribuído, mecanismos de cache, estratégias de replicação e sharding influenciam profundamente o design do sistema. O arquiteto precisa determinar como os dados serão armazenados, acessados e integrados entre componentes.
- ❑ Volume, variedade e veracidade: Com o crescimento do volume de dados (Big Data) e diversidade de fontes, arquitetos devem considerar tecnologias especializadas (data warehouses, lakes, streaming). Questões de desempenho em acesso a dados, escalabilidade de armazenamento, consistência vs. disponibilidade (CAP theorem) e governança de dados (segurança, privacidade) tornam-se parte das decisões arquiteturais.



LEIS FUNDAMENTAIS DA ARQUITETURA DE SOFTWARE

- ❑ Lei 1 - “Tudo é trade-off”: Em arquitetura de software, não existem soluções perfeitas, apenas diferentes compensações. Toda decisão melhora algum aspecto às custas de outro. Se algo parece não ter trade-off, provavelmente os trade-offs não foram identificados. Arquitetos devem estar sempre conscientes do que estão otimizando e que preço estão pagando por isso.
- ❑ Lei 2 - “Por quê é mais importante que Como”: Entender e registrar a motivação (porquês) por trás das decisões arquiteturais é mais importante do que o detalhe de implementação de como o sistema foi montado. O rationale das decisões guia futuras evoluções e permite a outros compreenderem a arquitetura. Em essência: saber por que algo foi feito supera saber apenas o que foi feito.



ATÉ A PRÓXIMA AULA



@eldermoraes